

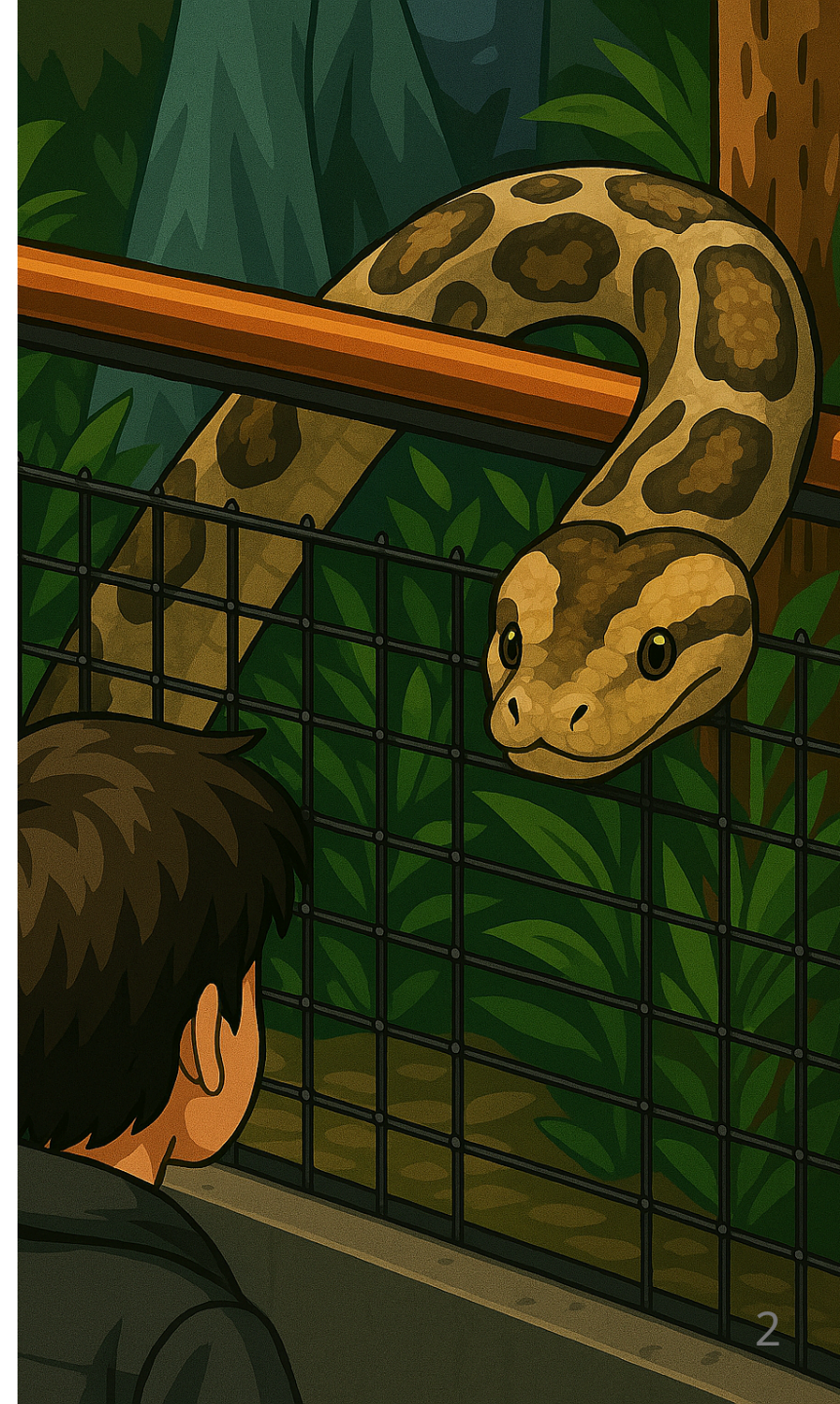
Python - POO, notions avancées

BTS CIEL



Sommaire

- Encapsulation et propriété dérivée
- Polymorphisme et héritage



Encapsulation et propriété dérivée

Voici une classe modélisant l'identité d'une personne :

```
class Person:  
  
    def __init__(self, firstname, lastname, age, nationality):  
        self.firstname = firstname  
        self.lastname = lastname  
        self.age = age  
        self.nationality = nationality
```

Encapsulation et propriété dérivée

On aimerait ajouter une information permettant de savoir si la personne est majeure ou non.

```
class Person:

    def __init__(self, firstname, lastname, age, nationality):
        self.firstname = firstname
        self.lastname = lastname
        self.age = age
        self.nationality = nationality

if __name__ == "__main__":
    person = Person("Jean", "Dupont", 65, "Française")

    if person.?????:
        print("Jean Dupont est majeur.")
```


Encapsulation et propriété dérivée

On pourrait ajouter un **attribut** :

```
class Person:

    def __init__(self, firstname, lastname, age, nationality, is_an_adult):
        self.firstname = firstname
        self.lastname = lastname
        self.age = age
        self.nationality = nationality
        self.is_an_adult = is_an_adult

if __name__ == "__main__":
    person = Person("Jean", "Dupont", 65, "Française", True)

    if person.is_an_adult:
        print("Jean Dupont est majeur.")

    # Mais cela ne garantit pas la cohérence des données :
    person.is_an_adult = False
    if not person.is_an_adult:
        print("Jean Dupont a 65 ans mais n'est pas majeur.")
```

Encapsulation et propriété dérivée

Parfois ce problème est plus subtil :

```
class Person:

    def __init__(self, firstname, lastname, age, nationality, is_an_adult):
        self.firstname = firstname
        self.lastname = lastname
        self.age = age
        self.nationality = nationality
        self.is_an_adult = is_an_adult

if __name__ == "__main__":
    person = Person("Jean", "Dupont", 17, "Française", True)

    person.age = 65

    # Mince, on a oublié de mettre à jour "is_an_adult"
    if person.is_an_adult:
        print("Jean Dupont est majeur.")
```

Encapsulation et propriété dérivée

Pour éviter ce type d'erreur et surtout les **incohérences de données**, il est nécessaire de mieux **modéliser** l'information.

Une bonne pratique consiste à **ne pas stocker une donnée dérivée**, mais à la **calculer à partir des données primitives**.

Encapsulation et propriété dérivée

Il est possible d'utiliser une **méthode** :

```
class Person:

    def __init__(self, firstname, lastname, age, nationality):
        self.firstname = firstname
        self.lastname = lastname
        self.age = age
        self.nationality = nationality

    def is_an_adult(self):
        return self.age >= 18

if __name__ == "__main__":
    person = Person("Jean", "Dupont", 17, "Française")

    person.age = 65

    # OK : toujours cohérent
    if person.is_an_adult():
        print("Jean Dupont est majeur.")
```


Encapsulation et propriété dérivée

L'inconvénient de cette solution est le **recalcul systématique** de la donnée dérivée.

Si le calcul devient plus complexe, cela peut impacter les performances :

```
class File:

    def __init__(self, content):
        self.content = content

    def checksum(self):
        return hashlib.sha256(self.content.encode()).hexdigest()
```

Encapsulation et propriété dérivée

Afin de garder une syntaxe simple **tout en contrôlant le calcul**, Python propose le décorateur `@property`.

Il permet d'utiliser une méthode **comme un attribut**, c'est ce que l'on appelle une **propriété** :

```
class Person:

    def __init__(self, firstname, lastname, age, nationality):
        self._age = age
        self.nationality = nationality
        self._is_an_adult_cache = None

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        self._age = value
        self._is_an_adult_cache = None

    @property
    def is_an_adult(self):
        if self._is_an_adult_cache is None:
            self._is_an_adult_cache = self._age >= 18

        return self._is_an_adult_cache
```

Encapsulation et propriété dérivée

Principes fondamentaux à retenir :

- **Encapsulation** : le code à l'extérieur de la classe ne connaît pas la manière dont la classe calcule la propriété `is_an_adult`. Il manipule un **attribut en apparence**, alors qu'il s'agit en réalité d'une **logique interne maîtrisée** (via une méthode ou une propriété).
- **Propriété dérivée** : la propriété `is_an_adult` est **calculée à partir de données internes** (`age`, voire `nationality`). Elle ne doit **pas être stockée directement**, mais **déduite** afin de garantir la **cohérence des données**.

Optimisation (optionnelle) : si le calcul devient coûteux, on peut mettre en place un **cache**, tout en veillant à l'**invalidier** dès que les données sources changent.

Encapsulation et propriété dérivée

Exercice : prendre en compte la nationalité

```
class Person:

    def __init__(self, firstname, lastname, age, nationality):
        self._age = age
        self.nationality = nationality
        self._is_an_adult_cache = None

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        self._age = value
        self._is_an_adult_cache = None

    @property
    def is_an_adult(self):
```